

Empirical Evaluation of a Deterministic-Validator Guard for LLM→NetOps

Generate→Verify→Repair Pipelines (Revised)

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a paired confirmatory experiment that measures the operational impact of inserting a deterministic network-configuration validator into an LLM-driven generate→verify→repair (GVR) loop for intent→configuration NetOps tasks. Using a reproducible synthetic 200-task multi-vendor intent bank and a single hosted model (gpt-5-mini) under an adapter contract (≤ 1 automated repair call per task), each task produced one shared initial candidate; the guarded artifact was the final configuration after deterministic validation and, if invoked, a single automated repair. Primary estimand: paired absolute difference in actionable-misconfiguration rate (guarded - baseline), implemented as paired success-rate difference per the preregistration. Results (N=200 pairs): baseline valid-config count = 199/200 (0.995), guarded valid-config count = 200/200 (1.000); absolute paired change (guarded - baseline) = 0.005 (0.5 percentage points). Exact McNemar two-sided test $p = 1.0$; paired bootstrap 95% CI = [0.0, 0.015] (10,000 resamples, seed 1729). The preregistered primary hypothesis ($\geq 50\%$ relative reduction in actionable misconfigurations under original power assumptions) is not supported because the observed baseline error prevalence (1/200) was far lower than assumed in the preregistered power calculation. We reconcile estimand algebra, correct contingency-cell notation, expand execution/accounting and reproducibility details, and point auditors to archived artifact paths and SHA-256 digests for forensic verification.

I. INTRODUCTION

Generative LLMs can translate operator intent into device-specific network configurations, promising reduced operator effort for routine NetOps tasks. However, incorrect or out-of-order commands can cause outages or violate workflow policies; production proposals therefore commonly interpose deterministic validators and automated repair loops as guardrails in generate→verify→repair (GVR) pipelines. Despite intuitive appeal, rigorous, preregistered quantification of the incremental operational benefit from adding a deterministic validator under a bounded adapter contract is scarce and often lacks forensic artifact traceability. We preregistered study `cnsmspec2026_gvr_v1_prereg_001` (SHA-256 = 5cb8a30815eacf0a3ca659d1a54fbaa71218e5ce57c0b13e26580995c78a6084) to answer: How much does integrating a deterministic network validator into an LLM-driven GVR pipeline reduce actionable misconfiguration rates (paired comparison) on a 200-task synthetic multi-vendor NetOps task bank, and what residual error types persist after automated repairs? Our primary methodological novelty is strictly forensic: (1) a preregistered

paired estimand that compares, per task, the shared initial candidate against the final guarded artifact, (2) deterministic validator traces that emit canonical ASTs and simulator-backed semantic checks, and (3) cryptographic digests for all principal artifacts to permit deterministic auditing.

II. RELATED WORK

Deterministic network verification and runtime validators (header-space analysis; Kazemian et al.; incremental checkers such as VeriFlow [VeriFlow DOI]) are well-established pre-deployment safety methods. Recent LLM→NetOps evaluations (e.g., NetConfEval [NetConfEval DOI], Lira et al.) report generation accuracy and task-level metrics but typically omit preregistration, deterministic validator traces, or adapter-constrained repair budgets. Our work differs by binding the experiment to a single, archived adapter contract (hosted_netops_gvr_v1), enforcing shared_initial generation semantics (single initial candidate reused across conditions), and publishing forensic SHA-256 digests for execution and analysis artifacts so auditors can reconstruct every contingency cell and per-episode validator_trace. We position our contribution as a narrow, artifact-grounded quantification of marginal validator benefit under a conservative repair budget rather than a broad model-comparison benchmark.

III. METHODOLOGY

Preregistration and estimand. We preregistered a paired binary design with N=200 intent→configuration tasks stratified by planned vendor family (planned strata: Cisco-like N=66; Juniper-like N=67; RFC-neutral N=67) and defined the primary_estimand as paired_success_rate_difference_guarded_minus_baseline (success = non-actionable configuration). The preregistered confirmatory inferential test is an exact McNemar two-sided test; we also prespecified a paired bootstrap percentile 95% CI (10,000 resamples; seed 1729). Full preregistration (`cnsmspec2026_gvr_v1_prereg_001`) SHA-256 = 5cb8a30815eacf0a3ca659d1a54fbaa71218e5ce57c0b13e26580995c78a6084 Adapter contract and generation semantics. Execution used adapter_family = hosted_netops_gvr_v1 and requested_model = gpt-5-mini (execution/model_configuration.json SHA-256 = a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073597fdd82) The adapter enforced shared_initial_generation semantics (independent_condition_generation = false), producing a

Statistical analysis pipeline. The analysis executor `paired_binary_analysis_v1` consumed `execution/raw_results.jsonl` and produced the contingency table (`n_11,n_10,n_01,n_00`), exact McNemar two-sided p-value, and paired bootstrap percentile CI for the absolute paired difference (`bootstrap_seed = 1729`; `resamples = 10,000`). All analysis artifacts (`analysis/paired_contingency_table.csv` SHA-256 = 658051ba..., `analysis/condition_summary.csv` SHA-256 = 86ab6b56...) are archived. We preserved the preregistered multiplicity plan (one primary confirmatory hypothesis; secondary/exploratory tests labeled as exploratory).

Operational interpretation. Under the constrained execution (synthetic 200-task bank; gpt-5-mini; deterministic_netops_validator_v5; single repair attempt), the guarded pipeline yielded a measurable but numerically small absolute benefit because the baseline generator already produced extremely few actionable errors (1/200). The contingency counts ($n_{11} = 199$, $n_{10} = 1$, $n_{01} = 0$, $n_{00} = 0$; analysis/paired_contingency_table.csv SHA-256 = 658051bae1f10d90aed1728b8d20dd3700a5694b68b02f1b0c118c45d9414c45d9414c1b0) make the pattern clear: nearly every initial candidate satisfied the full intent constraints, and the single observed improvement (the one n_{10} pair) was the only case in which the validator+repair changed an actionable outcome. Provider-call accounting confirms this operational pathway: provider_call_audit shows hosted_model_call_event_count = 201 with stage_counts = {"shared_initial_generation": 200, "repair": 1} (analysis/deterministic_reconciliation.json SHA-256 = 8a2b85f8...), meaning the adapter invoked the guarded pipeline to repair exactly once and that repair produced the solitary guarded improvement.

Implications for deployment and cost-benefit calibration. Practically, the marginal utility of deterministic validators depends on four interacting factors: (1) baseline LLM error prevalence in the target workload, (2) validator semantic coverage (which dictates whether errors are detected), (3) repair budget and automation policy (how many automated attempts or human interventions are allowed), and (4) the operational cost of an undetected actionable error. When baseline error prevalence is very low, high-cost, deep validators or heavy automation may yield diminishing returns on average even though they can prevent the occasional costly outage. Conversely, in deployments where baseline failures or rare catastrophic errors are more common, or where the cost of a single failure is extremely high, investing in broader validator coverage and larger repair budgets can be justified. Our observed run (1→0) demonstrates that a guarded GVR loop can correct rare actionable mistakes, but it does not by itself justify broad conclusions about the economic value of such guards without workload-specific cost modeling.

Statistical and decision-theoretic perspective. The two-sided exact McNemar p-value ($p = 1.0$) and the bootstrap CI [0.0, 0.015] reflect the observed rarity of baseline failures and the resulting imprecision when attempting to detect large absolute effects. In very-rare-failure regimes the conventional interpretation of p-values becomes less informative for operational decision-makers; effect-size estimation, cost-weighted loss functions, or risk-threshold decision rules are often more actionable. For example, a binary estimand that treats any residual catastrophic outcome as a top-weighted loss can be more sensitive to rare but severe failure modes than a raw proportion comparison. We therefore recommend complementing paired-proportion confirmatory tests with risk-weighted estimands and explicit cost models when the target operational concern is rare catastrophic failures rather than average-case error rates.

Design lessons and recommended adaptations. The pre-registered power analysis targeted a large absolute paired reduction (≈ 0.15) under assumed `baseline_error > 0.2`; given the observed `baseline_error = 0.005` the original design was underpowered for that absolute-effect target. Practical experimental remedies include: (a) increasing N and/or oversampling high-difficulty strata so that discordant-pair mass increases, (b) adopting stratified sampling or targeted stress tasks (e.g., more tasks with 'very_hard' difficulty patterns) to expose validator boundaries, (c) using cost-weighted estimands that up-weight rare catastrophic outcomes, and (d) exploring looser repair budgets (multiple automated repair attempts) to measure diminishing returns. A compact post-hoc sensitivity note in `analysis/analysis_log.jsonl` illustrates that, under `baseline_error = 0.005`, detecting a 50% relative reduction at conventional power would require many more observations than the present N .

Failure-mode and validator-coverage diagnostics. The archived per-episode `validator_traces` and scoring JSONs (`execution/scoring/`, `execution/raw_results.jsonl` SHA-256 = 9eeaa0c8...) enable fine-grained failure-mode taxonomy: syn-

tactic/parse errors, sequencing/dependency violations, and simulator-detected semantic/policy violations. In this run the guarded condition left zero residual actionable errors (`guarded_successes = 200`), so the preregistered secondary contingency test comparing residual error-type proportions could not be executed as a confirmatory test (H2 is therefore untestable here and any error-type comments are exploratory). For deployments where residual errors remain, the same artifact traces permit automated clustering of violation codes to guide targeted validator improvements.

Practical auditability and reviewer requests. Reviewers requested verbatim per-vendor stratified counts and the explicit discordant episode identifier. Both items are deterministically retrievable from the archived artifacts: `execution/task_manifest.jsonl` (SHA-256 = 4b28215e8a611feda50f6284058083b9ffcc2f5881f557081ce9e9f890f1c78a) and `execution/raw_results.jsonl` (SHA-256 = 9eeaa0c87db4742c807240b5ee18f934aaf45848350b3840f92af401bcb68d0) and the per-episode scoring JSONs under `execution/scoring/` (full hash manifest available in `execution/execution_manifest.json`). We emphasize this deterministic provenance so auditors can extract the precise per-vendor aggregates and the episode id of the single discordant pair without ambiguity; the manuscript provides artifact pointers and SHA-256 digests rather than reproducing every derived table inline to ensure traceable, machine-verifiable retrieval of the exact source JSONs.

VI. LIMITATIONS

- Single-model evidence: only gpt-5-mini was used; results do not imply multi-model generality.
- Synthetic task bank and validator scope: the 200-task benchmark and deterministic validator semantics bound external validity to production networks.
- Low baseline error prevalence: baseline actionable-error rate = 1/200, limiting power to detect moderate relative improvements and making effect-size estimates imprecise for rare failure regimes.
- Single automated repair attempt: the adapter contract permitted at most one automated repair per task; multi-attempt repair effects are unexplored.
- Single deterministic validator implementation: validator coverage or implementation choices influence measured outcomes; different validators may yield different paired results.
- Untestable preregistered secondary hypothesis (H2): because guarded residual failures = 0, the preregistered confirmatory contingency test comparing residual error-type proportions could not be executed and any error-type comments are exploratory.
- Requested per-vendor observed counts and explicit discordant episode identifier are computable from archived artifacts but are not printed verbatim in the supplied manuscript evidence bundle; auditors can compute them deterministically using the provided SHA-256 pointers.

VII. CONCLUSION

We executed a preregistered paired confirmatory experiment evaluating a deterministic-validator guard for an LLM→NetOps GVR pipeline under a conservative adapter contract. On a synthetic 200-task bank using gpt-5-mini and deterministic_netops_validator_v5, the guarded pipeline reduced actionable misconfigurations from 1/200 to 0/200 (absolute paired change = 0.005). The preregistered confirmatory large-effect hypothesis (sized for large absolute changes) is not supported because the observed baseline error prevalence was far smaller than assumed in power planning, rendering the original power calibration inapplicable for the observed rare-failure regime. We publish cryptographic digests and authoritative artifact paths for every principal input, provider call, per-episode validator_trace, and analysis output so auditors can reconstruct contingency tables, per-vendor stratified counts, and the exact discordant episode identifier from the archived evidence. Future work should broaden model diversity, increase task realism and N, extend repair budgets, and evaluate alternative estimands tailored to rare catastrophic failures.

DISCLOSURE STATEMENT

Disclosure Statement (mandatory). Initial master prompt immutable reference: provenance/master_prompt.txt SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39d15ba1
Preregistration: cnsmsspec2026_gvr_v1_prereg_001 SHA-256 = 5cb8a30815eac0a3ca659d1a54fbaa71218e5ce57c0b13e26580990077821529
Declared model and configuration: execution/model_configuration.json (requested_model = gpt-5-mini) SHA-256 = a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd0735077821529
Execution raw results and task manifest: execution/raw_results.jsonl SHA-256 = 9eeaa0c87db4742c807240b5ee18f934aaf45848350b3840f92af401bcb68d02
execution/task_manifest.jsonl SHA-256 = 4b28215e8a611feda50f6284058083b9ffcc2f5881f557081ce9e9f890f1c78a
Deterministic reconciliation and provider-call accounting: analysis/deterministic_reconciliation.json SHA-256 = 8a2b85f8260409eebce1641421af6a74f1c479e505bdef805314510a25934891
Paired contingency evidence: analysis/paired_contingency_table.csv SHA-256 = 658051bae1f10d90aed1728b8d20dd3700a5694b68b02f1b0c118c45d941440
Condition summary (success rates): analysis/condition_summary.csv SHA-256 = 86ab6b56e3ec10aa54aa08480a8e1ef31b64d79bf22bc99dac1d8ed00628a68a.
Missingness summary: analysis/missingness_summary.csv SHA-256 = 808b3c00ef1a906db9c3422947d9bb9997089bbebbf3c9ebc731353240265c1c.
Analysis executor inputs: analysis/results.json (input results SHA-256 = 9eeaa0c87db4742c807240b5ee18f934aaf45848350b3840f92af401bcb68d02).
Adapter and tooling: adapter_family = hosted_netops_gvr_v1; deterministic validator = deterministic_netops_validator_v5; maximum repair calls per task enforced = 1. Provider-call accounting explicit: provider_call_audit shows hosted_model_call_event_count = 201 and stage_counts

= {"shared_initial_generation": 200, "repair": 1}, which explains model_calls_used = 201 = 200 shared_initial + 1 repair (see execution/model_configuration.json SHA-256 above and deterministic_reconciliation.json SHA-256 above). Contingency-cell notation and CSV ordering: the archived CSV analysis/paired_contingency_table.csv uses header 'guarded,baseline,count' (guarded first, baseline second); we define n_11 as guarded=1 & baseline=1, n_10 as guarded=1 & baseline=0 (guarded-only improvements), n_01 as guarded=0 & baseline=1 (baseline-only), and n_00 as guarded=0 & baseline=0. Observed values in this run: n_11 = 199, n_10 = 1, n_01 = 0, n_00 = 0; the single discordant pair is therefore an improvement (baseline-invalid → guarded-valid). Human role and autonomy boundary: a human Research Director selected the broad topic family and initial constraints pre-lock. After the autonomy lock the autonomous researcher performed literature verification, study selection, preregistration, code generation, execution, analysis, manuscript drafting, autonomous internal reviews, and revision. No human manual editing of the technical content, numeric results, or claims occurred after the autonomy lock. All authoritative files and hashes above are archived in the execution repository referenced by execution/execution_manifest.json and are the source of truth for the corrected contingency labeling, per-vendor stratified accounting, and analysis outputs. Reviewer requests unresolved in-manuscript (but computable by (1) a verbatim per-vendor stratified numeric table (Cisco-like / Juniper-like / RFC-neutral) and (2) the explicit discordant episode identifier string. Both values are derivable from execution/task_manifest.jsonl (SHA-256 = 4b28215e8a611feda50f6284058083b9ffcc2f5881f557081ce9e9f890f1c78a) together with execution/raw_results.jsonl (SHA-256 = 9eeaa0c87db4742c807240b5ee18f934aaf45848350b3840f92af401bcb68d02); episode scoring JSONs under execution/scoring/ (full hash manifest in execution/execution_manifest.json); because the value appears verbatim in the supplied manuscript evidence bundle text we provide these artifact pointers rather than invent numeric summaries or episode ids in the analysis/condition_summary.csv where those values are not present verbatim.

REFERENCES

- [1] <https://seerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.228.5064>
- [2] <https://doi.org/10.1145/2342441.2342452>
- [3] <https://doi.org/10.1145/3656296>
- [4] <https://doi.org/10.1109/mcom.001.2400368>